UNIVERSIDAD DE GRANADA

ETSIIT INFORMÁTICA Y TELECOMUNICACIÓN



UNIVERSIDAD DE GRANADA

Departamento de Ciencias de la Computación e Inteligencia Artificial

Metaheurísticas

Guión de Prácticas

Práctica 1: Técnicas de Búsqueda de Poblaciones para el Problema de Portfolio

Curso 2025-26

Tercero en Grado en Ingeniería Informática

1 OBJETIVOS

El objetivo de esta práctica es estudiar el funcionamiento de las *Técnicas de Búsqueda Local y de los Algoritmos Greedy* en la resolución del Problema de Portfolio descrito en las transparencias del Seminario 2. Para ello, se requerirá que el estudiante adapte los siguientes algoritmos a dicho problema:

- Algoritmo Greedy básico.
- Algoritmo Aleatorio.
- Algoritmos de Búsqueda Local (BL).

El estudiante deberá comparar los resultados obtenidos con las estimaciones existentes para el valor de los óptimos de una serie de casos del problema.

La práctica se evalúa sobre un total de **1.5 puntos**, distribuidos de la siguiente forma:

- BL (**0.75 puntos**).
- Random **(0.25 puntos).**
- Greedy (**0.5 puntos**).

La fecha límite de entrega será el **el domingo 29 de marzo de 2026** antes de las 23:55 horas. La entrega de la práctica se realizará por internet a través del espacio de la asignatura en PRADO.

2 TRABAJO A REALIZAR

El estudiante deberá de desarrollar los distintos algoritmos al problema planteado. Los métodos desarrollados serán ejecutados sobre una serie de casos del problema. Se realizará un estudio comparativo de los resultados obtenidos y se analizará el comportamiento de cada algoritmo en base a dichos resultados. **Este análisis influirá decisivamente en la calificación final de la práctica**.

En las secciones siguientes se describen los aspectos relacionados con cada algoritmo a desarrollar y las tablas de resultados a obtener.

3 PROBLEMA Y CASOS CONSIDERADOS

3.1 Introducción al Problema del Portfolio

El problema del portfolio es un problema de interés planteado en el mundo de las finanzas: determinar la mejor asignación de capital entre activos para maximizar el retorno esperado, pero intentando minimizar el riesgo, y diversificando. Existen múltiples versiones más complejas (con valores fijos de acciones, limitar el número de acciones y/o compañías, fijar un riesgo fijo máximo, coste por transacción, optimización dinámica,...). En este enfoque vamos a querer optimizar el porcentaje de dinero que queremos invertir en cada una de las acciones disponibles, manteniendo restricciones sobre los porcentajes mínimos y máximos que se puede invertir por acción. Dichos porcentajes una vez asignado se mantendrán todo el periodo.

Más en detalle, queremos obtener el conjunto de pesos w_i que cumple:

- La suma de los pesos sea 1: $\sum_{i=1}^{n} w_i \approx 1$ (pongo $\approx$ porque con valores reales las comparaciones se hacen con un cierto margen de error, como 10^{-8}.
- Cada valor esté en un rango limitado: $(w_i == 0) \vee (lo \leq w_i \leq hi), \forall i \in 1..N$

Y el problema de optimización consta de una serie de N acciones, y por cada día t se ha calculado el retorno obtenido μ_{ti}, medido como el ratio de mejora, ya calculado según la ecuación 1.

$$\mu_{ti} = \frac{V_{ti} - V_{(t+1)i}}{V_{ti}} \tag{1}$$

Un valor positivo indica que ha habido mejora, y un valor negativo indica que se ha perdido dinero ese día.

A partir de dicha matriz de valores se puede calcular:

Media por acción $\overline{\mu_i} = \left(\sum_{t=1}^{T} \mu_{it} \right) /T$

Covarianza : $\Sigma_{ij} = \frac{1}{T} \sum_{t=1}^{T} (\mu_{ti} - \overline{\mu i}) \cdot (\mu_{tj} - \overline{\mu j})$

Beneficio obtenido : $\sum_i w_i \cdot \left(\sum_{t=1}^{t} \log(1 + \mu_{it}) \right) /T$

El beneficio obtenido se calcula usando logaritmos ya que así el cálculo acumulado es más sencillo, y el sumar 1 es para evitar problemas por los valores negativos.

La ecuación final a optimizar es indicada en la ecuación 2, en donde se hace una combinación entre el beneficio por cada acción y el riesgo, medido a partir de la matriz de covarianza:

$$\text{fitness}(w) = Beneficio(w) - \lambda \cdot Riesgo(w) \tag{2}$$

Donde se puede definir el beneficio y el riesgo en las ecuaciones 3 y 4, respectivamente:

$$Beneficio(w) = \sum_i w_i \cdot \left(\sum_t log(1 + \mu_{it}) \right) /T \tag{3}$$

$$Riesgo(w) = \sum_i \sum_j w_i \cdot \Sigma_{ij} \cdot w_j \tag{4}$$

Se usará un valor $\lambda = 500$.

Nota: En notación vectorial, el riesgo también se podría definir como $Riesgo(w) = w^\top \Sigma\, w$.

3.2 Conjuntos de Datos Considerados

Para abordar este problema, en vez de usar un benchmark estándar, se han obtenido los datos reales de distintas acciones durante el periodo 2015-2025. El número de días depende de los festivos. Además, se han eliminando aquellas empresas que no se mantuviese en bolsa durante todo el periodo con el mismo nombre.

Los ficheros finales obtenidos son:

ibex_35_data.csv Empresas del IBEX 35 (españolas), un total de 30 empresas durante 2813 días. En rango de valores permitido es $lo = 0{,}005, hi = 0{,}08$.

syp_100_data.csv Empresas del S&P 100 (índice de 100 mayores empresas de EE.UU.), un total de 97 empresas durante 2765 días. El rango de valores permitido es $lo = 0{,}005, hi = 0{,}05$.

syp_500_data.csv Empresas del S&P 500 (índice de 500 mayores empresas de EE.UU.), un total de 457 empresas durante 2765 días. El rango de valores permitido es $lo = 0{,}005, hi = 0{,}02$.

El formato de todos es CSV, en el que la primera columna, Date, es el día, y el resto de columnas son las distintas empresas, identificadas por siglas. El valor para cada empresa y día ya es directamente la mejora relativa μ_{ti}.

3.3 Generación Aleatoria de una solución

La generación de una solución es sencilla:

- Escoger un número de ceros de forma aleatoria (entre 1 y N-1).

- Para el resto de valores asignar un valor aleatorio entre lo y hi.
- Normalizar: $w_i = \frac{w_i}{\sum w_i}$

Puede tener que repararse, según el apartado siguiente.

3.4 Reparación de una solución

Un inconveniente que presenta esta representación del problema, es que a veces puede generarse soluciones que no cumplen las condiciones.

La condición para que se cumpla la suma sería fácil, si no se cumple se puede normalizar $w_i = \frac{w_i}{\sum w_i}$. Sin embargo, esa transformación puede implicar que los pesos ya no estén en el rango [lo,hi], por lo que puede ser necesario hacer lo siguiente:

- Si no se ha hecho antes, normalizar el rango $w_i = \frac{w_i}{\sum w_i}$.
- Luego asegurarse de nuevo que esté en el rango: $w_i \leftarrow 0$ si $w_i < lo$, y $w_i \leftarrow hi$ si $w_i > hi$.
- Comprobar si la suma de pesos es mayor o mayor que 1.
- Si es menor que 1, se eligen pesos mayores que 0 y menores que hi, y se divide la diferencia entre esos pesos sumándole un valor (manteniendo el rango). Si no es suficiente, se pueden incrementar pesos iguales a 0 (elegidos de forma aleatoria).
- Si fuese mayor que 1, aunque creo que no es posible si los pasos anteriores se hicieron bien, se pueden escoger pesos mayores que lo, y se divide la diferencia entre esos pesos restándole un valor (manteniendo el rango).

Un tema importante, para que la normalización pueda funcionar, es necesario que los valores lo y hi lo permitan. Por eso hemos usado valores distintos para cada conjunto.

4 DETERMINACIÓN DE LA CALIDAD DE UN ALGORITMO

El modo habitual de determinar la calidad de un algoritmo de resolución aproximada de problemas de optimización es ejecutarlo sobre un conjunto determinado de instancias y comparar los resultados obtenidos con los mejores valores conocidos para dichas instancias, si fuesen conocidos.

Además, los algoritmos pueden tener diversos parámetros o pueden emplear diversas estrategias. Para determinar qué valor es el más adecuado para un parámetro o saber la estrategia más efectiva los algoritmos también se comparan entre sí.

La comparación de los algoritmos se lleva a cabo fundamentalmente usando dos criterios, la *calidad* de las soluciones obtenidas y el *tiempo de ejecución* empleado para conseguirlas. Además, es posible que los algoritmos no se comporten de la misma forma si se ejecutan sobre un conjunto de instancias u otro.

Por otro lado, a diferencia de los algoritmos determinísticos, los algoritmos probabilísticos se caracterizan por la toma de decisiones aleatorias a lo largo de su ejecución. Este hecho implica que un mismo algoritmo probabilístico aplicado al mismo caso de un problema pueda comportarse de forma diferente y por tanto proporcionar resultados distintos en cada ejecución.

Cuando se analiza el comportamiento de un algoritmo probabilístico en un caso de un problema, se desearía que el resultado obtenido no estuviera sesgado por una secuencia aleatoria concreta que pueda influir positiva o negativamente en las decisiones tomadas durante su ejecución. Por tanto, resulta necesario efectuar varias ejecuciones con distintas secuencias probabilísticas y calcular el resultado medio (y a veces la desviación típica) de todas las ejecuciones para representar con mayor fidelidad su comportamiento.

Dada la influencia de la aleatoriedad en el proceso, es recomendable disponer de un generador de secuencia pseudoaleatoria de buena calidad con el que, dado un valor semilla de inicialización, se obtengan números en una secuencia lo suficientemente grande (es decir, que no se repitan los números en un margen razonable) como para considerarse aleatoria. En el espacio de PRADO se

puede encontrar una implementación en lenguaje C++ de un generador aleatorio de buena calidad (*random.hpp*).

Como norma general, el proceso a seguir consiste en realizar un número de ejecuciones diferentes de cada algoritmo probabilístico considerado para cada caso del problema. Es necesario asegurarse de que se realizan diferentes secuencias aleatorias en dichas ejecuciones. Así, el valor de la semilla que determina la inicialización de cada secuencia deberá ser distinto en cada ejecución y estas semillas deben mantenerse en los distintos algoritmos (es decir, la semilla para la primera ejecución de todos los algoritmos debe ser la misma, la de la segunda también debe ser la misma y distinta de la anterior, etc.). Por simplificar y facilitar la reproducibilidad, se usará la misma semilla para todos los casos de uso. Para mostrar los resultados obtenidos con cada algoritmo en el que se hayan realizado varias ejecuciones, se suelen construir tablas que recojan los valores correspondientes a estadísticos como el **mejor** y **peor** resultado para cada caso del problema, así como la **media** y la **desviación típica** de las ejecuciones. También se pueden emplear descripciones más representativas como los boxplots, que proporcionan información de todas las ejecuciones realizadas mostrando mínimo, máximo, mediana y primer y tercer cuartil de forma gráfica. Finalmente, se suelen construir unas tablas globales con los resultados agregados que mostrarán la calidad del algoritmo en la resolución del problema desde un punto de vista general.

5 ALGORITMOS A COMPARAR

5.1 Algoritmo Aleatorio

El algoritmo *aleatorio* se basa en generar **10000** soluciones de forma totalmente aleatoria, y devolver la mejor solución encontrada.

5.2 Algoritmo *Greedy*

Aunque el algoritmo *Greedy* es un algoritmo que conoce información del problema, y no es un algoritmo justo para comparar, vamos a aplicar un greedy sencillo, similar al que aplicaríamos para un problema de la mochila.

Calculamos para cada empresa una métrica heurística:

$$heur(i) = \left(\sum_t log(1 + \mu_{it})/T \right) - \lambda \cdot (\sum_t \Sigma_{ti})/T \tag{5}$$

Una vez calculado escogemos:

1. Ordenar las empresas en orden decreciente por su heurística, y guardarlo en un vector *companies*.
2. $w_i \leftarrow 0, \forall i \in [0, T]$.
3. Mientras $sum(w_i) < 1$
   a) $c \leftarrow first(companies)$.
   b) $companies \leftarrow companies - \{c\}$.
   c) $w[c] \leftarrow min(hi, 1 - sum(w_i))$.

De esta forma, intentamos seleccionar en todo momento la empresa con mayor heurística, hasta que no podamos seguir añadiendo más valores.

5.3 Algoritmo de Búsqueda Local (BL)

Este algoritmo es un algoritmo de la literatura, más complejo.

Se aplica una búsqueda *primero mejor*, en el que se generan el vecindario definido como indica la ecuación 6.

$$neigh(sol) = [(p1, p2), \forall p1 \in 1 : N, p2 \in 1 : N] \quad (6)$$

Una vez generado el vector, se desordena, y se van aplicando el cambio, en el que se quita a la posición i-ésima un % de su valor, y se incrementa a la posición j-ésima. Se ve con detalle en la ecuación 7.

$$change(sol, i, j) = [\cdots, sol_{i-1}, (1 - ratio) \cdot sol_i, sol_{i+1}, \cdots, sol_{j-1}, sol_j + ratio \cdot sol_i, sol_{j+1}, \cdots] \quad (7)$$

En particular, se usa un ratio=0.4, ya que es el valor más recomendado en la literatura.

5.4 Determinando la calidad de un algoritmo

El modo habitual de determinar la calidad de un algoritmo de resolución aproximada de problemas de optimización es ejecutarlo sobre un conjunto determinado de instancias y comparar los resultados obtenidos con los mejores valores conocidos para dichas instancias, si fuesen conocidos.

Además, los algoritmos pueden tener diversos parámetros o pueden emplear diversas estrategias. Para determinar qué valor es el más adecuado para un parámetro o saber la estrategia más efectiva los algoritmos también se comparan entre sí.

La comparación de los algoritmos se lleva a cabo fundamentalmente usando dos criterios, la *calidad* de las soluciones obtenidas y el *tiempo de ejecución* empleado para conseguirlas. Además, es posible que los algoritmos no se comporten de la misma forma si se ejecutan sobre un conjunto de instancias u otro.

Por otro lado, a diferencia de los algoritmos determinísticos, los algoritmos probabilísticos se caracterizan por la toma de decisiones aleatorias a lo largo de su ejecución. Este hecho implica que un mismo algoritmo probabilístico aplicado al mismo caso de un problema pueda comportarse de forma diferente y por tanto proporcionar resultados distintos en cada ejecución.

Cuando se analiza el comportamiento de un algoritmo probabilístico en un caso de un problema, se desearía que el resultado obtenido no estuviera sesgado por una secuencia aleatoria concreta que pueda influir positiva o negativamente en las decisiones tomadas durante su ejecución. Por tanto, resulta necesario efectuar varias ejecuciones con distintas secuencias probabilísticas y calcular el resultado medio (y a veces la desviación típica) de todas las ejecuciones para representar con mayor fidelidad su comportamiento.

Dada la influencia de la aleatoriedad en el proceso, es recomendable disponer de un generador de secuencia pseudoaleatoria de buena calidad con el que, dado un valor semilla de inicialización, se obtengan números en una secuencia lo suficientemente grande (es decir, que no se repitan los números en un margen razonable) como para considerarse aleatoria. En el espacio de PRADO se puede encontrar una implementación en lenguaje C++ de un generador aleatorio de buena calidad (*random.hpp*).

Como norma general, el proceso a seguir consiste en realizar un número de ejecuciones diferentes de cada algoritmo probabilístico considerado para cada caso del problema. Es necesario asegurarse de que se realizan diferentes secuencias aleatorias en dichas ejecuciones. Así, el valor de la semilla que determina la inicialización de cada secuencia deberá ser distinto en cada ejecución y estas semillas deben mantenerse en los distintos algoritmos (es decir, la semilla para la primera ejecución de todos los algoritmos debe ser la misma, la de la segunda también debe ser la misma y distinta de la anterior, etc.). Por simplificar y facilitar la reproducibilidad, se usará la misma semilla para todos los casos de uso. Para mostrar los resultados obtenidos con cada algoritmo en el que se hayan realizado varias ejecuciones, se suelen construir tablas que recojan los valores correspondientes a estadísticos como el **mejor** y **peor** resultado para cada caso del problema, así como la **media** y la **desviación típica** de las ejecuciones. También se pueden emplear descripciones más representativas como los boxplots, que proporcionan información de todas las ejecuciones realizadas mostrando mínimo, máximo, mediana y primer y tercer cuartil de forma gráfica. Finalmente, se suelen construir unas tablas globales con los resultados agregados que mostrarán la calidad del algoritmo en la resolución del problema desde un punto de vista general.

En el Portfolio consideraremos 50 ejecuciones con semillas de generación de números aleatorios diferentes para cada algoritmo en cada conjunto de datos. Esto quiere decir que un algoritmo cualquiera, se ejecutará 50 veces por cada conjunto de datos (con semillas distintas, y las mismas para todos los algoritmos), por lo que supondrá un total de 50×6=300 ejecuciones por algoritmo.

El resultado final de las medidas de validación de las tablas será calculado como la media de los 50 valores obtenidos para cada conjunto de datos y restricciones.

5.5 Tablas y Gráficas de Resultados a Obtener

Se diseñará una tabla para cada algoritmo (greedy, Random, BL) y cada fichero en donde se recojan los resultados de la ejecución de dicho algoritmo en los conjuntos de datos considerados. Tendrá la misma estructura que las Tablas 1, 2, y 3. En dichas tablas se puede ver el valor de fitness sobre el conjunto histórico (de 2015 a 2024), una proyección de dicho fitness sobre cómo sería aplicando los pesos al año 2025, así como su beneficio acumulado sobre dicho año. También se muestra el total de evaluaciones promedio de cada algoritmo (1 en el Greedy), y el tiempo medido en segundos. Todos los valores tendrán 2 o 3 decimales, no más.

Algoritmo	**2015-2024**	**2025**		**Evaluaciones**	**Tiempo**
	Fitness	**Fitness**	**Beneficio**		**(segundos)**
Greedy	valor	valor	valor	1	valor
Random	valor	valor	valor	valor	valor
BL	valor	valor	valor	valor	valor

Tabla 1: Resultados de Ibex 35 (Medias)

Algoritmo	**2015-2024**	**2025**		**Evaluaciones**	**Tiempo**
	Fitness	**Fitness**	**Beneficio**		**(segundos)**
Greedy	valor	valor	valor	valor	valor
Random	valor	valor	valor	valor	valor
BL	valor	valor	valor	valor	valor

Tabla 2: Resultados de S&P 100 (Medias)

Algoritmo	**2015-2024**	**2025**		**Evaluaciones**	**Tiempo**
	Fitness	**Fitness**	**Beneficio**		**(segundos)**
Greedy	valor	valor	valor	valor	valor
Random	valor	valor	valor	valor	valor
BL	valor	valor	valor	valor	valor

Tabla 3: Resultados de S&P 500 (Medias)

Adicionalmente, se deberá de realizar para cada ejemplo una visualización en boxplot mostrando la distribución de datos de cada algoritmo sobre la medida de fitness sobre el periodo 2015-2024. Un ejemplo sería la gráfica de la Figura 1.

A partir de los datos mostrados en estas tablas, el estudiante realizará un análisis de los resultados obtenidos, **que influirá significativamente en la calificación de la práctica**. En dicho análisis se deben comparar los distintos algoritmos en términos de calidad de las soluciones y tiempo requerido para producirlas. Por otro lado, se puede analizar también el comportamiento de los algoritmos en algunos de los casos individuales que presenten un comportamiento más destacado.

6 DOCUMENTACIÓN Y FICHEROS A ENTREGAR

En general, la **documentación** de ésta y de cualquier otra práctica será un fichero pdf que deberá incluir, al menos, el siguiente contenido:

a) Portada con el número y título de la práctica (con el nombre del problema), el curso académico, el nombre, DNI y dirección e-mail del estudiante, y su horario de prácticas.

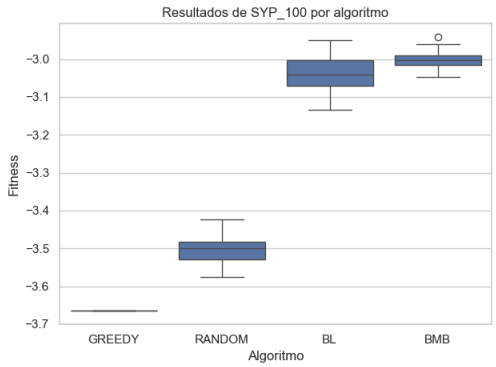


Figura 1: Resultados de Fitness en 2015-2024 para cada algoritmo en el S & Y 100

b) Índice del contenido de la documentación con la numeración de las páginas.

c) **Breve** descripción/formulación del problema (**máximo 1 página**). Podrá incluirse el mismo contenido repetido en todas las prácticas presentadas por el estudiante.

d) Breve descripción de la aplicación de los algoritmos empleados al problema (**máximo 4 páginas**): Todas las consideraciones comunes a los distintos algoritmos se describirán en este apartado, que será previo a la descripción de los algoritmos específicos. Incluirá por ejemplo la descripción del esquema de representación de soluciones y la **descripción en pseudocódigo (no código)** de la función objetivo y los operadores comunes.

e) Descripción en **pseudocódigo** de la **estructura del método de búsqueda** y de todas aquellas **operaciones relevantes** de cada algoritmo. Este contenido, específico a cada algoritmo se detallará en los correspondientes guiones de prácticas. El pseudocódigo **deberá forzosamente reflejar la implementación/ el desarrollo realizados** y no ser una descripción genérica extraída de las transparencias de clase o de cualquier otra fuente. La descripción de cada algoritmo no deberá ocupar más de **2 páginas**.

   Para esta primera práctica, se incluirán al menos las descripciones en pseudocódigo de:

   - Para el algoritmo *greedy*, aparte del esquema general, la función heurística.
   - Para el algoritmo BL, el métodos de exploración del entorno, el operador de generación de vecino, la factorización de la BL si fuese el caso, y la generación de soluciones aleatorias.

f) Breve explicación de la **estructura del código de la práctica**, incluyendo un pequeño **manual de usuario describiendo el proceso para que el profesor de prácticas pueda compilarlo (usando un sistema automático como make o similar) y cómo ejecutarlo, dando algún ejemplo de ejecución**.

g) Experimentos y análisis de resultados:

   - Descripción de los casos del problema empleados y de los valores de los parámetros considerados en las ejecuciones de cada algoritmo (**incluyendo las semillas utilizadas**).
   - Resultados obtenidos según el formato especificado.
   - Análisis de resultados (**Máximo: 4 páginas sin contar las figuras**). El análisis deberá estar orientado a **justificar** (según el comportamiento de cada algoritmo) **los resultados** obtenidos en lugar de realizar una mera "lectura" de las tablas. Se valorará la inclusión de otros elementos de comparación tales como gráficas de convergencia, boxplots, análisis comparativo de las soluciones obtenidas, representación gráfica de las soluciones, etc.

h) Referencias bibliográficas u otro tipo de material distinto del proporcionado en la asignatura que se haya consultado para realizar la práctica (en caso de haberlo hecho).

Aunque lo esencial es el contenido, también debe cuidarse la presentación y la redacción. **La documentación nunca deberá incluir listado total o parcial del código fuente**.

En lo referente al **desarrollo de la práctica**, se entregará una carpeta llamada **software** que contenga una versión ejecutable de los programas desarrollados, así como el código fuente implementado o los ficheros de configuración del framework empleado. El código fuente o los ficheros de configuración se organizarán en la estructura de directorios que sea necesaria y deberán colgar del directorio *src* en el raíz. Junto con el código fuente, hay que incluir los ficheros necesarios para construir los ejecutables según el entorno de desarrollo empleado (tales como *.prj, makefile, *.ide, etc.). En este directorio se adjuntará también un pequeño fichero de texto de nombre LEEME que contendrá breves reseñas sobre cada fichero incluido en el directorio. Es importante que los programas realizados puedan leer los valores de los parámetros de los algoritmos desde fichero, es decir, que no tengan que ser recompilados para cambiar éstos ante una nueva ejecución. Por ejemplo, la semilla que inicializa la secuencia pseudoaleatoria debería poder especificarse como un parámetro más.

En el caso de que en el lenguaje de programación elegido se haya ofrecido un API, deberá de **obligatoriamente implementarlo siguiendo el API ofrecido**. Si no fuese el caso, se adaptará el API recomendado.

El fichero pdf de la documentación y la carpeta software serán comprimidos en un fichero .zip etiquetado con los apellidos y nombre del estudiante (Ej. Pérez Pérez Manuel.zip). Este fichero será entregado por internet a través del espacio de la asignatura en PRADO.

7 MÉTODO DE EVALUACIÓN

Al principio de la práctica se ha indicado la puntuación máxima que se puede obtener por cada algoritmo y su análisis. **La inclusión de trabajo voluntario** (desarrollo de variantes adicionales, experimentación con diferentes parámetros, prueba con otros operadores o versiones adicionales del algoritmo, análisis extendido, etc.) **podrá incrementar la nota final** por encima de la puntuación máxima definida inicialmente, o compensar parcialmente errores en la práctica.

En caso de que el comportamiento del algoritmo en la versión implementada/ desarrollada no coincida con la descripción en pseudocódigo o no incorpore las componentes requeridas, se podría reducir hasta en un 50 % la calificación del algoritmo correspondiente.